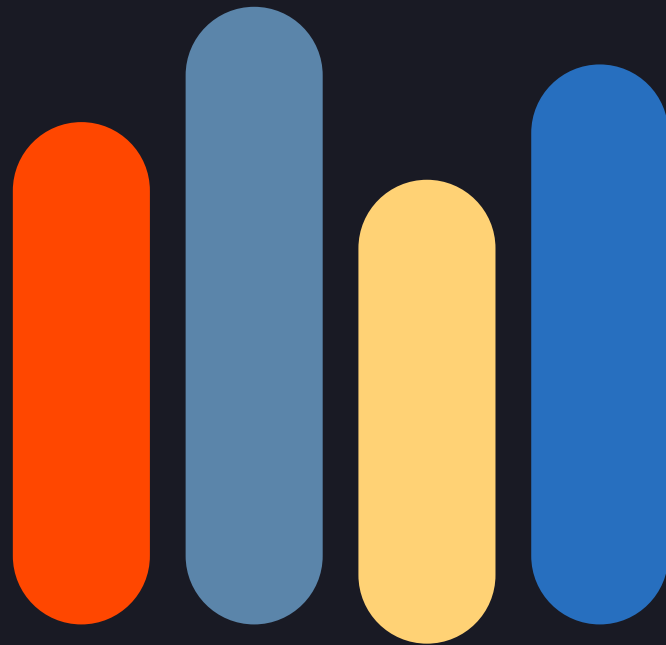


Refactoring in Java

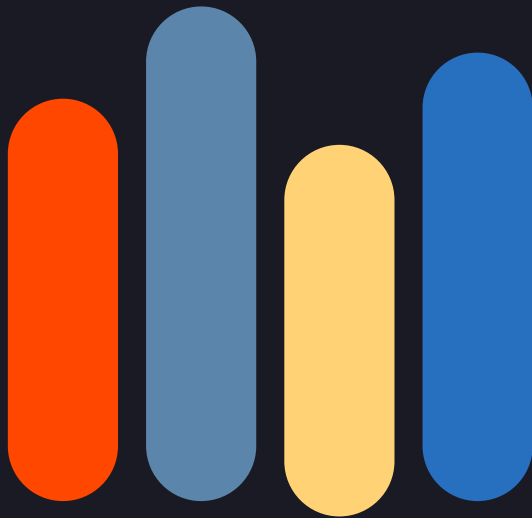
Five techniques in 30 minutes

Java SE 21 | Maven | JUnit 5



Agenda

- What refactoring is (and isn't)
- The safety net: tests stay green
- Challenge 1: Rename and replace magic numbers
- Challenge 2: Extract method
- Challenge 3: Introduce parameter object
- Challenge 4: Replace type code with sealed types
- Challenge 5: Replace loops with streams
- Wrap-up





Refactoring changes the structure of code without changing what it does.

Same inputs, same outputs. Easier to read, test and change.

Adapted from Martin Fowler, Refactoring (2nd ed.)

The safety net

Tests are green before you start, and green after every step.

1

Run the tests

All 15 pass

2

**Make one small
change**

Use the IDE where you can

3

Run the tests again

Red? Undo, take a smaller
step

4

Commit

Then repeat

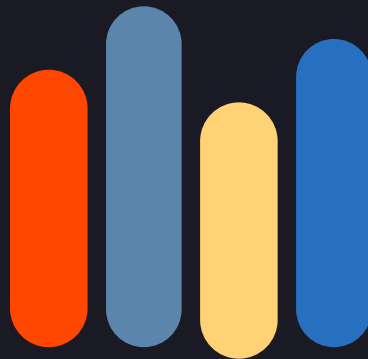
The scenario: OrderProcessor

It works and the tests prove it. These business rules must not change.

Rule	Value
Customer discounts	Silver 10%, Gold 20%, Staff 30%
Promo code WELCOME10	10.00 off the goods, never below zero
Standard shipping	4.99, free when goods are 50.00 or more
Express delivery	+ 9.99
Delivery outside Ireland (IE)	+ 15.00
VAT	23% on goods plus shipping

The five challenges

- Names and constants
- Small methods
- Parameter objects
- Sealed types
- Streams



CHALLENGE 1 OF 5

Rename and replace magic numbers

The smell

Mysterious names and magic numbers. What are t, l, d and r? What does 0.23 mean?

The technique

Rename Variable, Rename Method, Replace Magic Number with Symbolic Constant

Before

```
double r = d + s;  
r = r + r * 0.23;
```

After

```
static final double VAT_RATE = 0.23;  
  
double beforeVat = goods + shipping;  
return beforeVat * (1 + VAT_RATE);
```

CHALLENGE 2 OF 5

Extract method

The smell

Long method. calculateTotal does five jobs in one block.

The technique

Extract Method

Before

```
public double calculateTotal(...) {  
    // 35 lines of loops, ifs  
    // and arithmetic  
}
```

After

```
double goods = subtotal(items);  
goods = applyTierDiscount(goods, tier);  
goods = applyPromoCode(goods, promo);  
double total = goods + shippingCost(...);  
return roundToCents(addVat(total));
```

CHALLENGE 3 OF 5

Introduce parameter object

The smell

Long parameter list. Two String arguments can be swapped and still compile.

The technique

Introduce Parameter Object (using a record)

Before

```
calculateTotal(String tier,  
               List<Item> items, String country,  
               boolean express, String promo)
```

After

```
record OrderRequest(String tier,  
                   List<Item> items,  
                   String countryCode,  
                   boolean expressDelivery,  
                   String promoCode) { }
```

```
calculateTotal(OrderRequest order)
```

CHALLENGE 4 OF 5

Replace type code with sealed types

The smell

Primitive obsession. A typo like "GLOD" compiles and silently gives no discount.

The technique

Replace Type Code with Subclasses, using sealed interfaces and pattern matching (Java 21)

Before

```
if (t.equals("GOLD")) {  
    d = d - d * 0.2;  
} else if (t.equals("SILVER")) {  
    ...  
}
```

After

```
sealed interface CustomerTier { ... }  
  
return switch (tier) {  
    case Regular r -> 0.0;  
    case Silver s -> SILVER_DISCOUNT;  
    case Gold g -> GOLD_DISCOUNT;  
    case Staff s -> STAFF_DISCOUNT;  
};
```

CHALLENGE 5 OF 5

Replace loops with streams

The smell

Duplicated code. Four methods repeat the same loop-and-accumulate pattern.

The technique

Replace Loop with Pipeline

Before

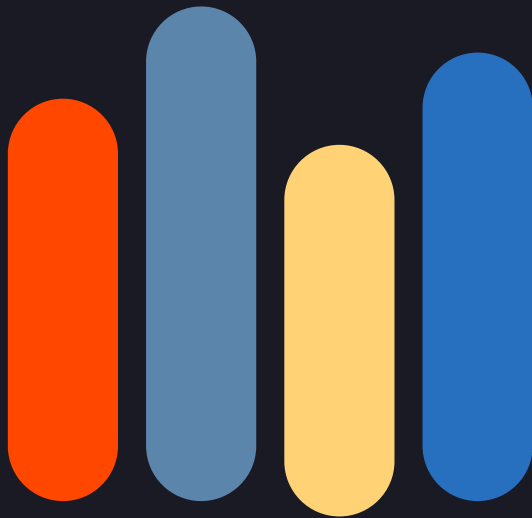
```
List<String> out = new ArrayList<>();
for (Item i : l) {
    if (i.quantity() >= 10) {
        out.add(i.name());
    }
}
```

After

```
return items.stream()
    .filter(i -> i.quantity() >= BULK_QUANTITY)
    .map(Item::name)
    .toList();
```

What we covered

- Rename and replace magic numbers
- Extract method
- Introduce parameter object with a record
- Replace type code with sealed types and pattern matching
- Replace loops with stream pipelines



Do

- Take small steps and run the tests after each one
- Let the IDE do rename and extract
- Use records and sealed types so the compiler catches mistakes
- Keep refactoring and new features in separate commits

Don't

- Refactor code that has no tests
- Change behaviour and call it a refactor
- Rewrite everything at once
- Use double for money in production (look up BigDecimal)

Thank you

Questions?

neueda [